07_data_b

Root Cause Analysis

Q26. Walk me through how you do structured root-cause analysis on a metric drop, versus just "looking at the data."

Ideal answer. Structured RCA replaces eyeballing with a decomposition-first discipline: I never start from "what looks suspicious," I start from an arithmetic identity that the total *must* obey, then attribute the change to its mathematically exhaustive parts. For a funnel-rate drop I first localize *which step* moved (each step rate is `numerator_sessions / upstream_sessions`), then decompose *that* step's change across segments into mix, rate, and interaction so I know whether traffic composition or in-segment behavior moved, and only then drill the largest *rate* contributors into a hypothesis tree. The discipline is: localize -> decompose -> attribute -> verify, where each stage narrows the search space before the next, so I never chase a segment that is merely a composition artifact. The failure mode I am avoiding is the "largest absolute delta" reflex - picking the segment that fell the most in raw count, which is dominated by your biggest-volume segment and is wrong roughly half the time.

Why Helios demonstrates it. This is the exact Monitor -> Decompose -> Diagnose pipeline: `detect_anomaly` localizes, `decompose_change` attributes, and the Diagnose agent runs a best-first hypothesis tree over the rate contributors - and the offline benchmark scores this at >=85% root-cause accuracy versus <=45% for the naive largest-delta baseline.

Follow-ups. How do you decide when to stop drilling the tree? / What stops the LLM from inventing a plausible-sounding cause that the data does not support? / How would this change if the metric were revenue rather than a rate?

Q27. Explain mix-shift versus rate-change to me as if I were a non-technical PM, then give me the math.

Ideal answer. Plain version: imagine overall conversion dropped. Two very different things could have happened - either *each kind of visitor* converts worse than before (rate-change, a real behavior problem you can fix), or *each kind still converts the same* but you simply got more low-converting visitors, say a cheap paid-social surge (mix-shift, a traffic-composition story, not a product problem). They demand opposite actions, so conflating them wastes analyst-weeks. The math: for an aggregate rate `R = Σ_i w_i·r_i` (segment volume share times segment rate), the change splits exactly as `ΔR = Σ Δw_i·r_i(t0)` (mix) `+ Σ w_i(t0)·Δr_i` (rate) `+ Σ Δw_i·Δr_i` (interaction). The decomposition is exhaustive and exact - the three terms sum to the total change

- which is what lets me say with confidence "78% of the drop is rate, not mix." That exactness is also how you dissolve Simpson's paradox: a metric can fall in aggregate while rising in every segment purely because the mix shifted toward low-rate segments.

Why Helios demonstrates it. `stats-mcp.decompose_change` computes these three terms deterministically in Python (never in token space), and the Diagnose agent is hard-wired to drill rate-effects before mix-effects - because rate is real behavior and mix is often a composition artifact.

Follow-ups. When is a mix-shift itself the actionable finding rather than a distraction? / How do you handle the interaction term when you summarize for an exec? / What if a segment appears in t1 but not t0 (zero baseline weight)?

Q28. What is a hypothesis tree in RCA and how do you keep it from exploding combinatorially?

Ideal answer. A hypothesis tree is a structured search where the root is the observed anomaly and each node is a candidate explanation that, if true, would account for some quantified share of the movement; children refine a parent into more specific sub-causes (e.g., "checkout-step drop" -> "mobile checkout drop" -> "mobile Safari checkout drop"). You keep it from exploding by making it *best-first and quantified*: you only expand the branch carrying the largest unexplained rate-effect, you prune any branch whose attributable share is below a materiality floor, and you cap depth once a node explains enough of the delta to be decision-grade. This is fundamentally different from a breadth-first crawl of every dimension combination, which is both expensive and noisy. The governing principle is that the decomposition gives every node a *number*, so expansion is driven by magnitude of unexplained variance, not by curiosity.

Why Helios demonstrates it. The Diagnose agent (Opus) runs exactly this best-first hypothesis-tree RCA, expanding the highest-rate-effect branch and SQL-verifying each node via semantic-mcp before it commits - so the tree is both bounded and grounded, never a free-form LLM ramble.

Follow-ups. How do you set the materiality floor without it being arbitrary? / What if two branches each explain 40% - how do you report a split cause? / How do you avoid double-counting when child segments overlap?

Q29. Why are "5 Whys" and similar qualitative frameworks insufficient for analytics RCA?

Ideal answer. 5 Whys is a useful *framing* heuristic but it has two fatal gaps for analytics: it has no notion of *magnitude* (each "why" is asserted, not measured, so you can walk a chain that explains 3% of the drop while feeling complete), and it is single-threaded (it assumes one causal chain, when real funnel moves are usually several causes summing to the total). Quantified attribution fixes both: every candidate cause carries an attributable share derived from the decomposition,

the shares are mutually exclusive and sum to ~100% of the delta, and you rank by dollar or rate magnitude rather than by narrative plausibility. I still *use* the 5-Whys instinct to generate candidate causes, but I refuse to ship a cause without a number attached. The senior move is to treat qualitative frameworks as hypothesis generators and quantified decomposition as the adjudicator.

Why Helios demonstrates it. Principle 4 of the project - "a finding without a significance test, a dollar impact, and a recommended action is not a finding" - is enforced structurally: the Narrator literally cannot emit a cause that lacks a `decompose_change` share and a `significance_test` verdict behind it.

Follow-ups. Give me a case where the qualitatively obvious cause was quantitatively trivial. / How do you communicate that a cause is "real but immaterial"? / Where does 5 Whys still earn its keep in your workflow?

Q30. How do you distinguish a real behavioral change from a confounder?

Ideal answer. A confounder is a variable correlated with both your segment split and the outcome, such that an apparent in-segment rate change is actually a composition effect inside that segment. My defense is layered: first the mix/rate/interaction split already separates composition from behavior at the chosen segmentation; second, I re-decompose along a *plausibly-confounding* dimension (if a "Paid Social rate drop" survives controlling for device and new-vs-returning, it is more credible); third, I demand the rate-effect be statistically significant given the segment's sample size, so I am not chasing noise. The conceptual point is that no single decomposition proves causation - I am stacking refutation attempts, and a finding earns trust by *surviving* the obvious confounders rather than by being asserted. Honest caveat: on observational data I can never fully close the causal loop, which is why the output is a *designed experiment*, not a causal claim.

Why Helios demonstrates it. This is precisely the Critic agent's job - it runs adversarial refutation against four named failure modes (mix-confound, insufficient sample, seasonality, data-quality) and returns a verdict of PASS / DOWNGRADE / DROP, so a finding only ships if it survives the confound attack.

Follow-ups. Which confounder bites GA4 funnel analysis most often? / How do you choose which dimension to re-decompose against? / When do you accept a finding as DOWNGRADE rather than DROP?

Q31. How do you separate seasonality from a genuine anomaly?

Ideal answer. Seasonality is expected, recurring variation; an anomaly is a deviation from the seasonally-adjusted expectation. So the right test is never "is today lower than yesterday" - it is "is

today lower than what a model that already knows the weekly/holiday pattern predicted, by more than the prediction interval." I fit a forecast that captures weekly seasonality and holiday effects, take the residual, and only flag points whose residual exceeds a band scaled to historical noise. A naive week-over-week or day-over-day comparison will scream every Monday and every post-Black-Friday Tuesday. The maturity point is also maintaining a *known-events calendar* so that a promotion or a launch is treated as an explained deviation, not a mystery to diagnose.

Why Helios demonstrates it. `stats-mcp.forecast` (prophet/statsmodels) supplies the seasonally-aware expectation that `detect_anomaly` flags against, and the memory layer keeps seasonality and launch calendars so the Critic can DROP a finding that is just a known seasonal swing - directly relevant given the dataset's Nov-2020-to-Jan-2021 holiday window.

Follow-ups. How much history do you need before a seasonal model is trustworthy here? / How do you handle a one-off event with no prior precedent? / What is the cost of a false-positive anomaly to the analyst's trust in the system?

Q32. How do data-quality issues masquerade as anomalies, and how do you guard against them?

Ideal answer. Most "anomalies" in raw event data are tracking artifacts, not behavior: a tag deploy that drops `add_to_cart` events, a late-arriving partition that makes yesterday look low, a bot surge inflating sessions, or a consent change suppressing a channel. These produce textbook funnel drops that fool a naive system into prescribing a product fix for a measurement bug. My guard is a battery of cheap sanity checks before causal interpretation: reconcile totals against a canonical source, check for implausible step-rate values (>1 means broken monotonic flags), watch for a channel or device going abruptly to zero (a tracking signature, not a behavior signature), and confirm the partition is complete. The principle: rule out the instrument before you diagnose the patient.

Why Helios demonstrates it. Two structural guards - warehouse-mcp's mandatory `reconcile` (>0.5% drift fails the finding) and the `reached_*` max-downstream monotonic flags that make step rates impossible to exceed 1 - plus a data-quality bucket in the 50-scenario eval that includes pure data-quality artifacts the pipeline must label as such rather than diagnose.

Follow-ups. How would you detect a partial tag-drop that only affects mobile? / What is your reconciliation tolerance and why that number? / How do you keep the system from "crying wolf" on every late partition?

Q33. Give me your end-to-end RCA narrative for a single concrete anomaly.

Ideal answer. Say `session_conversion_rate` falls 12% week-over-week. (1) Localize: I decompose the overall rate into step rates and find the drop concentrates at `begin_checkout` →

`add_payment_info`, not upstream. (2) Decompose that step across `device_category x channel_group`: the split returns ~80% rate-effect, ~15% mix, ~5% interaction - so this is behavior, not composition. (3) Drill the largest rate contributor: mobile + a specific channel, and the hypothesis tree narrows to mobile checkout. (4) Significance: confirm the mobile rate delta clears its sample-size threshold. (5) Refute: control for new-vs-returning to kill the confound, and check the calendar to rule out seasonality and a tag deploy. (6) Quantify: price the surviving drop as dollars of revenue-at-risk via lost purchasing sessions x AOV. (7) Prescribe: a powered experiment on the mobile payment step. (8) Ship: a Decision Brief leading with the dollar number and the one recommended action. The whole point is that each step is a tool output, not a vibe.

Why Helios demonstrates it. This is the literal agent handoff chain - Monitor, Decompose, Diagnose, Critic, Prescribe, Narrator - passing a typed JSON "Finding" envelope, completing in under 5 minutes per run versus the 1-3 analyst-days the manual version costs.

Follow-ups. Where in that chain is the most likely place to be wrong? / How do you report it if the cause is genuinely split across two segments? / What would you do differently if you only had three days of history?

Business Analysis

Q34. How do you size the business impact of a funnel anomaly in dollars?

Ideal answer. Sizing converts a rate movement into money so it competes for attention on a common scale. The clean approach is counterfactual: estimate the conversions (or sessions) lost relative to the seasonally-adjusted expectation, then monetize them at the relevant value-per-conversion. For a checkout-rate drop: `revenue_at_risk = lost_purchasing_sessions x AOV`, where `lost_purchasing_sessions = affected_sessions x Δrate`. I am careful to (a) size only the *rate* portion of the move, not the mix portion, since mix-driven changes are often not recoverable by a product fix; (b) bound the estimate with the significance interval so it reads as a range, not false precision; and (c) annualize cautiously, flagging that the dataset is a 3-month window. A concrete sketch:

```sql
-- revenue-at-risk from a rate drop at one funnel step (illustrative)
SELECT
  affected_segment,
  upstream_sessions,
  (rate_t0 - rate_t1)                       AS delta_rate,
  upstream_sessions * (rate_t0 - rate_t1)   AS lost_purchasing_sessions,
  upstream_sessions * (rate_t0 - rate_t1) * aov AS revenue_at_risk_usd
FROM step_decomposition
WHERE effect_type = 'rate'    -- price behavior, not composition
ORDER BY revenue_at_risk_usd DESC;
```

Why Helios demonstrates it. Every finding carries a dollar impact by design (Principle 4), priced off the rate-effect from `decompose_change` and AOV from the governed registry - and the eval even labels dollar-at-risk, which is why `fct_daily_funnel` must carry `session_revenue` rather than dropping it.

Follow-ups. Why size only the rate portion and not the whole delta? / How do you turn a point estimate into a defensible range? / How would you adjust the AOV if the affected segment has a different basket than the overall average?

Q35. How do you prioritize a backlog of findings when you can't act on all of them?

Ideal answer. Prioritization needs a single comparable score, and I build it from impact, confidence, and effort. Impact is the dollar revenue-at-risk (or upside) already computed; confidence is how well the finding survived refutation and how tight its significance interval is; effort/cost is the implementation and experiment runtime. A defensible ranking multiplies expected dollar impact by a confidence weight and divides by effort - essentially an ICE/RICE variant where "Reach" and "Impact" collapse into the quantified dollar number rather than a 1-5 guess. The discipline that separates this from typical RICE is that the inputs are *measured*, not voted: impact comes from decomposition, confidence from the Critic's verdict and the significance test, and runtime from a power analysis. The failure mode I avoid is the loudest-stakeholder problem, where bets are ranked by conviction rather than by defensible expected value.

Why Helios demonstrates it. The Prescribe agent emits powered experiment cards, and prioritization combines the dollar revenue-at-risk with the Critic's PASS/DOWNGRADE verdict and experiment-mcp's `runtime_estimate` - so the backlog is ranked by defensible expected impact per unit of effort, not by argument.

Follow-ups. How do you weight a high-impact / low-confidence finding against a low-impact / high-confidence one? / What happens to DOWNGRADE findings in the ranking? / How do you stop the ranking from always favoring the biggest-volume segment?

Q36. Translate a metric move into an actual business decision for me.

Ideal answer. A metric move only matters once it changes a decision, so I force every diagnosis to terminate in a recommended action with an owner and an expected payoff. The structure is: here is what moved (the step and segment), here is why (the dominant rate-effect, refuted against confounders), here is what it costs (dollars of revenue-at-risk), and therefore here is what we should do and what we expect from it (a specific experiment with a powered sample size and an upside estimate). For the mobile-checkout example: "We are losing *~X/weektoamobilepayment – stepdropconcentratedinreturningusers*; *recommendanA/Bonasimplifiedmobilepaymentsheet*,

poweredtodetecta2ptlift, 14 − *dayruntime*, *expectedrecovery* Y." That is a decision a PM can approve or reject, not a chart they have to interpret. The senior point: the deliverable is a *decision*, and the metric is just the evidence.

Why Helios demonstrates it. The Narrator's Decision Brief leads with the dollar number and one prioritized action backed by a Prescribe experiment card - the entire product is built to close the "insight-to-action gap" rather than report the insight and stop.

Follow-ups. What do you do when the right action is "do nothing - it's a mix-shift we expected"? / How do you set the recommendation if the experiment would take longer than the window of opportunity? / Who is the owner of a recommendation and how do you track whether it was acted on?

Q37. How do you handle a metric that is "statistically significant but practically irrelevant," or the reverse?

Ideal answer. Significance and materiality are orthogonal and you must report both. A huge sample can make a 0.1pt rate change "significant" while it is worth almost nothing in dollars; conversely a chunky, expensive movement in a small segment may fail significance yet still deserve a watch-flag. My rule is to gate on *both* - significance establishes the effect is real, the dollar size establishes it is worth a decision - and to never let one stand in for the other. When something is significant but immaterial I explicitly label it "real but not worth acting on" so it does not consume a slot in the backlog; when it is material but underpowered I prescribe *collecting more data* (or a longer experiment) rather than acting blind. The honesty here reads as senior: I am refusing to let a p-value masquerade as importance.

Why Helios demonstrates it. Helios separates the two channels structurally - `stats-mcp.significance_test` decides "real," `decompose_change` plus AOV decides "material" - and the Critic can DOWNGRADE a statistically-significant-but-immaterial finding so it does not crowd out a bigger fish.

Follow-ups. What materiality threshold do you set and how do you justify it? / How do you explain a non-significant but expensive movement to an exec without alarming them? / When does "collect more data" become an excuse for inaction?

Q38. A stakeholder insists "conversion is down, fix it." How do you scope the real problem before committing analyst time?

Ideal answer. I resist jumping to a fix and instead run a fast triage that scopes the problem in minutes: is the drop real (versus seasonality or a late partition), where is it (which step), what kind is it (mix versus rate), how big is it (dollars), and is it even ours to fix (a tracking artifact?). Most "conversion is down" panics dissolve at this stage - it is a known seasonal dip, or a tag deploy, or

a paid-traffic surge diluting the mix rather than a product regression. Only the residual that survives triage earns a deep dive and a prescribed experiment. This protects the scarcest resource, analyst time, and it reframes the stakeholder conversation from "fix it" to "here is precisely what moved and whether it warrants action." The mature framing is that *scoping is the deliverable* for half of these requests.

Why Helios demonstrates it. This triage is exactly the Monitor -> Decompose -> Critic front of the pipeline, which can rule out seasonality, data-quality, and mix-shift before the expensive Diagnose/Prescribe stages ever run - turning a 1-3-day investigation into a <5-minute autonomous run.

Follow-ups. How do you push back on a stakeholder without sounding dismissive? / What is the cheapest check that kills the most false alarms? / How do you log the triage so the same panic doesn't get re-investigated next week?

Q39. How do you quantify the upside of a fix, not just the downside of the problem?

Ideal answer. Downside is the revenue-at-risk you can recover; upside is the realistic share of that you expect a specific intervention to recapture, bounded by the experiment's minimum detectable effect. I never assume a fix recovers 100% of the loss - I size the recoverable portion as `expected_uplift = affected_sessions x detectable_rate_lift x value_per_conversion`, where the detectable lift comes from the power analysis, and I present it as a range. This is what makes the prescription honest: the experiment is powered to detect the *smallest lift worth the effort*, and the upside is tied to that floor rather than to wishful thinking. The senior nuance is that you also subtract the experiment's own cost and opportunity cost, so the recommendation is net-positive expected value, not just "directionally good."

Why Helios demonstrates it. experiment-mcp's `power_analysis` and `runtime_estimate` set the detectable effect and duration, so each Prescribe card's upside is grounded in what the experiment can actually prove - not a fabricated recovery number.

Follow-ups. How do you set the minimum detectable effect - business floor or statistical convenience? / What if the powered runtime exceeds the seasonal window the opportunity exists in? / How do you account for novelty effects inflating early uplift?

Dashboarding

Q40. When do dashboards genuinely help, and when do they fail?

Ideal answer. Dashboards excel at *monitoring known questions*: tracking a small set of agreed KPIs over time, confirming the business is on-plan, and giving everyone a shared, governed number. They fail at *diagnosis* - the moment a metric moves, a dashboard can show you the *what*

but it cannot tell you *why*, because answering "why" requires an open-ended search through segment combinations that no fixed set of tiles can enumerate. The classic failure is the dashboard sprawl: dozens of charts, each answering a question nobody is currently asking, while the actual root-cause investigation still happens in an analyst's ad-hoc notebook over the next two days. So dashboards are a *pull* surface for surveillance of known metrics; they are structurally incapable of the open-ended causal search that RCA demands, and pretending otherwise is why "self-serve BI" so often fails to reduce analyst load.

Why Helios demonstrates it. Helios is explicitly *not* a dashboard - it occupies the diagnosis gap a dashboard cannot fill, running the open-ended mix/rate hypothesis search a fixed tile set never could, and treating conversation as a secondary drill-down rather than the product.

Follow-ups. Where would you still put a dashboard alongside Helios? / Why doesn't adding more dashboard filters solve the why-problem? / What's the maintenance cost of a 40-tile dashboard nobody reads?

Q41. Explain push versus pull in analytics delivery and why it matters.

Ideal answer. Pull means a human must remember to go look (open the dashboard, run the query) - which means insights are found only when someone happens to be looking and already suspects something. Push means the system surfaces the insight to the person *when it occurs*, unprompted. The why-it-matters: most anomalies in a pull model are caught late or never, because monitoring 28 metrics across 16 dimensions by hand is not something humans do consistently. A push model inverts the burden - the default is "you will be told when something material moves and why," so attention is spent on *deciding*, not on *hunting*. The tradeoff to manage is alert fatigue: a push system is only trusted if its precision is high, which is why suppression, materiality gating, and seasonality-awareness are not optional features but the cost of being allowed to push at all.

Why Helios demonstrates it. Helios's heartbeat is the *autonomous scheduled run* (push), not a queried dashboard (pull); the memory layer's suppression list and the Critic's seasonality/data-quality DROPs exist specifically to keep push precision high enough to be trusted.

Follow-ups. How do you tune the materiality floor to balance recall and alert fatigue? / What's the first thing that erodes trust in a push system? / When would a user still want to pull rather than be pushed to?

Q42. Why is your "Decision Brief" an anti-dashboard, and what does it contain?

Ideal answer. A dashboard hands you evidence and makes *you* do the synthesis; a Decision Brief does the synthesis and hands you a *decision*. It inverts the cognitive load. The brief leads with the conclusion - what moved, the dollar revenue-at-risk, and the single recommended action - then

provides the supporting chain underneath for anyone who wants to audit it: the decomposition shares, the significance verdict, the confounders ruled out, and the powered experiment design. It is the pyramid principle applied to analytics: answer first, evidence second. The reason this is the anti-dashboard is that its unit of delivery is a recommendation a leader can approve or reject in one read, whereas a dashboard's unit of delivery is raw charts that still require an analyst-in-the-loop to interpret. The discipline is ruthless: if the brief cannot be acted on, it failed, no matter how rich the underlying analysis.

Why Helios demonstrates it. The Narrator agent renders the Decision Brief via report-mcp's `render_brief`, leading with the dollar number and one prioritized, experiment-backed action - the literal end-state that distinguishes Helios from a BI tool.

Follow-ups. How do you keep the brief honest when the finding is ambiguous? / What goes "above the fold" versus in the audit trail? / How long should an exec spend reading one?

Q43. How do you design a dashboard that supports diagnosis rather than just reporting, if you had to build one?

Ideal answer. If forced to build a diagnostic dashboard, I'd design it around the decomposition workflow rather than around metric tiles: a top-line anomaly indicator against a seasonally-adjusted expectation, then a drill path that always shows the mix/rate/interaction split for the selected metric and lets you pivot the segmentation dimension, with every number traceable to a governed definition. The key design choice is to encode the *analysis path* (localize -> decompose -> drill) into the navigation, not to scatter independent charts. But I'd be honest in the interview that this is a *crutch* - it speeds a human through the standard path but still relies on the human to drive the search and stop at the right node, which is exactly the labor an autonomous system should remove. So I'd build it as a drill-down surface attached to an automated engine, not as the primary product.

Why Helios demonstrates it. This mirrors Helios's deliberate ordering - localize, decompose, drill rate-before-mix - and its decision to make conversation/drill-down a *secondary* surface bolted onto the autonomous run, never the main deliverable.

Follow-ups. What governed definitions would you enforce on every tile? / How do you prevent the drill path from becoming a 30-click maze? / Where's the line between a helpful drill-down and re-inventing the analyst's manual grind?

Q44. What's the strongest argument *for* dashboards, and how do you reconcile it with building an anti-dashboard?

Ideal answer. The strongest case for dashboards is shared situational awareness with a governed number: an org needs one trusted place where everyone sees the same KPIs, defined the same

way, updated reliably - that is real and valuable, and Helios does not replace it. I reconcile it by seeing them as different jobs: dashboards answer "are we on track" (monitoring known metrics, a solved problem), while Helios answers "why did we go off track and what do we do" (diagnosis and prescription, the unsolved problem). The mistake is asking a dashboard to do the second job by piling on filters until it becomes an unusable ad-hoc query tool. So I'm not anti-dashboard dogmatically - I'm against the *category error* of expecting a reporting surface to do causal diagnosis. The two coexist: governed monitoring up front, autonomous diagnosis behind it.

Why Helios demonstrates it. Helios deliberately scopes itself to the diagnosis/prescription gap and leans on the same governed definitions (the 28-metric / 16-dimension semantic registry) a good dashboard would use - so it complements monitoring rather than competing with it.

Follow-ups. Where exactly do you draw the boundary between the two tools? / How do you keep both honoring the same metric definitions? / Have you ever seen a dashboard succeed at diagnosis? What made it work or fail?

Analytics Workflows

Q45. Why does a governed metrics layer matter, and what breaks without one?

Ideal answer. A governed metrics layer is a single, versioned registry where every metric and dimension is defined exactly once, so "conversion rate" means the same thing in every query, brief, and dashboard. Without it you get *metric drift*: three analysts compute "conversion" three ways (sessions vs users in the denominator, engaged vs all sessions), the numbers disagree in a leadership meeting, and trust in the entire data function erodes. The deeper failure for an AI system is hallucination - an LLM left to write SQL will invent plausible column names and silently wrong joins. Governing the metrics means the machine can only *compose* from validated definitions, never author free SQL, which is the difference between an analyst tool you can trust unattended and a demo. A subtle but critical rule the layer enforces: compute rates as `SUM(num)/SUM(den)` after grouping, never as an average of per-segment ratios - the latter quietly reintroduces Simpson's paradox.

Why Helios demonstrates it. semantic-mcp is the *only* path to SQL, composing from `semantic_models.yml` (28 metrics, 16 dimensions, referential-integrity compiled, 1:1 with dbt MetricFlow) - which is how Helios hits 0 hallucinated columns and why an unknown metric name is a hard error, not a fallback to free SQL.

Follow-ups. How do you add a new metric without letting it drift? / Why is "average of ratios" specifically dangerous? / How do you version a metric definition when its meaning legitimately changes?

Q46. How do you make an analysis reproducible?

Ideal answer. Reproducibility means anyone can re-run the analysis and get the same numbers, which requires controlling four things: the data (a frozen or version-pinned snapshot, not a live shifting table), the definitions (the governed metric layer, so the SQL is regenerated identically), the computation (deterministic, seeded math rather than hand calculations or stochastic LLM arithmetic), and the parameters (the exact window, filters, and thresholds, captured as code/config). The enemy of reproducibility is hidden state - an analyst's local notebook with an un-pinned query and a mental note about which week they used. I push all four into governed, versioned artifacts so the run is a function of explicit inputs. The maturity point: an analysis you cannot reproduce is an anecdote, and an autonomous system that is not reproducible cannot be trusted to run unattended.

Why Helios demonstrates it. Determinism-where-it-matters is a core principle - all math runs seeded in real Python via stats-mcp (never token-space), SQL is regenerated from the versioned semantic registry, and the eval injects anomalies into a *frozen* GA4 copy so runs are exactly reproducible and CI-gated.

Follow-ups. Why route math through Python instead of letting the LLM compute it? / How does freezing the eval dataset help reproducibility? / What's the hardest source of hidden state to eliminate in practice?

Q47. The "analyst-time problem" - what is it and how does automation actually help?

Ideal answer. The analyst-time problem is that the scarcest resource in a data team is senior analyst attention, and it gets consumed by *repetitive, mechanical* RCA - the same localize/decompose/drill grind on every anomaly, costing 1-3 days each - leaving little time for the genuinely novel, strategic work only a human can do. Automation helps not by replacing the analyst but by *eliminating the mechanical 80%*: the system handles the deterministic decomposition, the significance testing, the dollar sizing, and the first-pass prescription, and escalates to the human only the ambiguous or strategic residual. The trap is automating the *easy* part (a chart) instead of the *expensive* part (the diagnosis), which is most BI tooling's mistake. Done right, automation shifts the analyst from *producing* the analysis to *adjudicating and acting on* it - higher leverage per hour.

Why Helios demonstrates it. Helios automates exactly the expensive part - the full Monitor-to-Narrator diagnosis chain in <5 minutes versus 1-3 analyst-days - and leaves the human the high-value act of approving the prescribed experiment, which is the right division of labor.

Follow-ups. Which part of RCA should *not* be automated and why? / How do you measure the analyst-time actually saved? / What's the risk of analysts deskilling if the machine does the

diagnosis?

Q48. What does a healthy analytics workflow look like end to end, and where does automation slot in?

Ideal answer. A healthy workflow is a governed pipeline: raw events land, get modeled into clean, tested marts with a layered transformation DAG, surface through a single semantic layer, and feed both monitoring (push alerts on materiality) and on-demand diagnosis - with every output traceable back to a governed definition and a reproducible run. Automation slots in at two places: continuous monitoring (so anomalies are detected without a human watching) and first-pass diagnosis (so the why-analysis is drafted before an analyst opens it). The human stays in the loop for adjudication, strategic framing, and the act of deciding. The principle binding it together is *governance flows downstream* - if the metric layer is the single source of truth, then alerts, diagnoses, briefs, and dashboards are all guaranteed consistent, and you have eliminated the reconciliation meetings that eat data teams alive.

Why Helios demonstrates it. Helios is that pipeline made concrete: GA4 -> dbt staging/intermediate/marts -> the `semantic_models.yml` layer -> the autonomous scheduled run -> the Decision Brief, with warehouse-mcp's reconcile gate enforcing consistency to <=0.5% the whole way down.

Follow-ups. Where is the single most leveraged place to add governance? / How do you keep the dbt DAG and the semantic layer from diverging? / What part of this workflow is hardest to get an org to adopt?

Q49. How do you build trust in an automated analytics system so people actually act on its output?

Ideal answer. Trust is earned by being *correct, transparent, and accountable*, and you have to engineer all three. Correctness: ground every number in governed definitions and deterministic math, and prove accuracy on a labeled benchmark rather than asserting it. Transparency: show the work - the decomposition shares, the significance verdict, the confounders ruled out - so a skeptic can audit any claim rather than being asked to take it on faith. Accountability: close the loop by tracking whether the system's recommendations, once acted on, actually worked - a system that grades itself in production is one you can trust to run unattended. The failure mode is the black-box oracle that is occasionally spectacularly wrong and offers no audit trail; one such miss destroys adoption permanently. So the hard part was never generating insights - it was making them correct and *trusted enough to act on*.

Why Helios demonstrates it. All three pillars are built in: governed-SQL grounding (semantic-mcp) + deterministic stats (stats-mcp) for correctness, the auditable Finding envelope and

Decision Brief for transparency, and the Critic plus the 85%-vs-45% offline benchmark and the did-the-fix-work action-tracking loop for accountability.

Follow-ups. What single piece of evidence convinces a skeptical analyst fastest? / How does the did-the-fix-work loop change behavior over time? / What would one high-profile wrong call do, and how do you recover trust?

Q50. Where does automation *not* belong in analytics - what do you deliberately keep human?

Ideal answer. Automation belongs on the mechanical, well-specified, repeatable parts - decomposition, significance testing, sizing, seasonality adjustment - and explicitly *not* on three things: causal claims on observational data (the machine can suggest, but only a designed experiment can confirm), strategic prioritization across competing business objectives (which weighs goals the data cannot see), and novel-situation judgment with no precedent to learn from. I deliberately keep the human as the adjudicator who approves prescriptions and owns the decision, because accountability for a business action should sit with a person. The senior framing is that I automate to *concentrate* human judgment on the decisions that deserve it, not to remove the human - an autonomous diagnosis engine whose output a human rubber-stamps is fine; one that auto-executes a pricing change is reckless. Knowing where to stop is itself the senior skill.

Why Helios demonstrates it. Helios deliberately *designs and sizes* experiments rather than auto-running live A/Bs - honest about the dataset being observational and historical - and the Decision Brief presents a recommendation for a human to approve, never an action it executes autonomously.

Follow-ups. Give me a decision you'd never let Helios make alone. / How do you encode "this needs a human" into the system? / Where's the line between autonomous diagnosis and autonomous action, and why does it matter?